

```
In [1]: !pip3 install pandas numpy matplotlib seaborn scikit-learn statsmodels xgboost
```

Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: pandas in /Users/anurag/Library/Python/3.9/lib/python/site-packages (2.3.3)

Requirement already satisfied: numpy in /Users/anurag/Library/Python/3.9/lib/python/site-packages (2.0.2)

Requirement already satisfied: matplotlib in /Users/anurag/Library/Python/3.9/lib/python/site-packages (3.9.4)

Requirement already satisfied: seaborn in /Users/anurag/Library/Python/3.9/lib/python/site-packages (0.13.2)

Requirement already satisfied: scikit-learn in /Users/anurag/Library/Python/3.9/lib/python/site-packages (1.6.1)

Requirement already satisfied: statsmodels in /Users/anurag/Library/Python/3.9/lib/python/site-packages (0.14.6)

Requirement already satisfied: xgboost in /Users/anurag/Library/Python/3.9/lib/python/site-packages (2.1.4)

Requirement already satisfied: lightgbm in /Users/anurag/Library/Python/3.9/lib/python/site-packages (4.6.0)

Requirement already satisfied: tensorflow in /Users/anurag/Library/Python/3.9/lib/python/site-packages (2.20.0)

Requirement already satisfied: scipy in /Users/anurag/Library/Python/3.9/lib/python/site-packages (1.13.1)

Requirement already satisfied: python-dateutil>=2.8.2 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from pandas) (2026.1.post1)

Requirement already satisfied: tzdata>=2022.7 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from pandas) (2026.2)

Requirement already satisfied: contourpy>=1.0.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (1.3.0)

Requirement already satisfied: cycler>=0.10 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (4.60.2)

Requirement already satisfied: kiwisolver>=1.3.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (1.4.7)

Requirement already satisfied: packaging>=20.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (26.2)

Requirement already satisfied: pillow>=8 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: importlib-resources>=3.2.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from matplotlib) (6.5.2)

Requirement already satisfied: joblib>=1.2.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from scikit-learn) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.1.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: patsy>=0.5.6 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from statsmodels) (1.0.2)

Requirement already satisfied: absl-py>=1.0.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (2.3.1)

Requirement already satisfied: astunparse>=1.6.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (25.12.19)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (0.7.0)

Requirement already satisfied: google_pasta>=0.1.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (18.1.1)

Requirement already satisfied: opt_einsum>=2.3.2 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (3.4.0)

Requirement already satisfied: protobuf>=5.28.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (6.33.6)

Requirement already satisfied: requests<3,>=2.21.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (2.32.5)

Requirement already satisfied: setuptools in /Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/site-packages (from tensorflow) (58.0.4)

Requirement already satisfied: six>=1.12.0 in /Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/site-packages (from tensorflow) (1.15.0)

Requirement already satisfied: termcolor>=1.1.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (3.1.0)

Requirement already satisfied: typing_extensions>=3.6.6 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (4.15.0)

Requirement already satisfied: wrapt>=1.11.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (2.1.2)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (1.80.0)

Requirement already satisfied: tensorboard~2.20.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (2.20.0)

Requirement already satisfied: keras>=3.10.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (3.10.0)

Requirement already satisfied: h5py>=3.11.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (3.14.0)

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorflow) (0.5.4)

Requirement already satisfied: charset_normalizer<4,>=2 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from requests<3,>=2.21.0->tensorflow) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from requests<3,>=2.21.0->tensorflow) (2.6.3)

Requirement already satisfied: certifi>=2017.4.17 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from requests<3,>=2.21.0->tensorflow) (2026.1.4)

Requirement already satisfied: markdown>=2.6.8 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorboard~2.20.0->tensorflow) (3.9)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorboard~2.20.0->tensorflow) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from tensorboard~2.20.0->tensorflow) (3.1.8)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.

9/site-packages (from astunparse>=1.6.0->tensorflow) (0.37.0)
 Requirement already satisfied: zipp>=3.1.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from importlib-resources>=3.2.0->matplotlib) (3.23.1)
 Requirement already satisfied: rich in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from keras>=3.10.0->tensorflow) (14.3.2)
 Requirement already satisfied: namex in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from keras>=3.10.0->tensorflow) (0.1.0)
 Requirement already satisfied: optree in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from keras>=3.10.0->tensorflow) (0.19.1)
 Requirement already satisfied: importlib-metadata>=4.4 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from markdown>=2.6.8->tensorboard~2.20.0->tensorflow) (8.7.1)
 Requirement already satisfied: markupsafe>=2.1.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from werkzeug>=1.0.1->tensorboard~2.20.0->tensorflow) (3.0.3)
 Requirement already satisfied: markdown-it-py>=2.2.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from rich->keras>=3.10.0->tensorflow) (3.0.0)
 Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from rich->keras>=3.10.0->tensorflow) (2.19.2)
 Requirement already satisfied: mdurl~0.1 in /Users/anurag/Library/Python/3.9/lib/python/site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.2)

```
In [2]: import os, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from math import radians, sin, cos, sqrt, atan2
from scipy.stats import poisson
from sklearn.calibration import calibration_curve
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression, PoissonRegressor
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import log_loss, confusion_matrix
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

warnings.filterwarnings("ignore")
tf.get_logger().setLevel("ERROR")

SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
```

```
tf.random.set_seed(SEED)

plt.style.use("seaborn-v0_8-whitegrid")
sns.set_context("notebook", font_scale=1.1)

WINDOW = 5
TSCV = TimeSeriesSplit(n_splits=5)
LABELS = [0, 1, 2] # Away=0, Draw=1, Home=2
```

```
/Users/anurag/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.p
y:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently
the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/
urllib3/urllib3/issues/3020
warnings.warn(
```

```
In [3]: SEASON_URLS = [
    "https://www.football-data.co.uk/mmz4281/2324/E0.csv",
    "https://www.football-data.co.uk/mmz4281/2425/E0.csv",
    "https://www.football-data.co.uk/mmz4281/2526/E0.csv",
]

COLS = [
    "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR",
    "HS", "AS", "HST", "AST", "HC", "AC", "HF", "AF",
    "B365H", "B365D", "B365A",
]

dfs = []
for url in SEASON_URLS:
    try:
        dfs.append(pd.read_csv(url, storage_options={"User-Agent": "Mozilla/
        print(f" Loaded {url.split('/')[-2]}")
    except Exception as e:
        print(f" Failed {url}: {e}")

df = pd.concat(dfs, ignore_index=True)[COLS].dropna().copy()
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
df = df.sort_values("Date").reset_index(drop=True)

print(f"\nTotal matches : {len(df)}")
print(f"Date range      : {df['Date'].min().date()} to {df['Date'].max().date()}")
print(f"Max nulls        : {df.isnull().sum().max()}")

Loaded 2324
Loaded 2425
Loaded 2526

Total matches : 1130
Date range    : 2023-08-11 to 2026-05-19
Max nulls     : 0
```

```
In [4]: stat_cols = ["FTHG", "FTAG", "HS", "AS", "HST", "AST", "HC", "AC", "HF", "AF"]

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Outcome bar chart
ax = sns.countplot(data=df, x="FTR", order=["H", "A", "D"], palette="pastel")
```

```

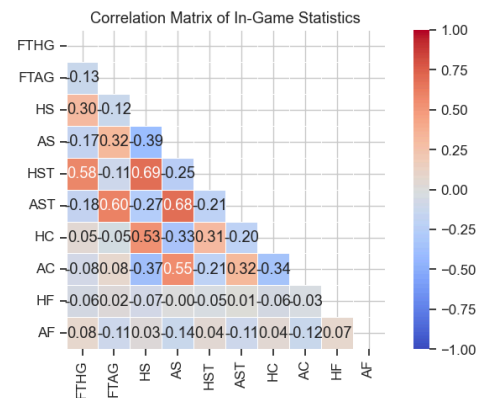
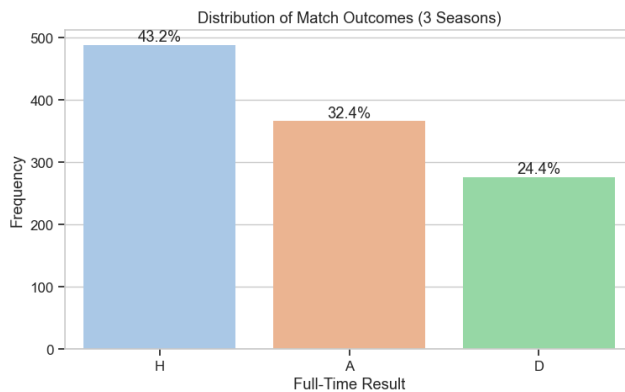
axes[0].set_title("Distribution of Match Outcomes (3 Seasons)")
axes[0].set_xlabel("Full-Time Result")
axes[0].set_ylabel("Frequency")
for p in ax.patches:
    pct = f"{100 * p.get_height() / len(df):.1f}%"
    axes[0].annotate(pct, (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom")

# Correlation heatmap
corr = df[stat_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True, linewidths=0.5, ax=axes[1])
axes[1].set_title("Correlation Matrix of In-Game Statistics")

plt.tight_layout()
plt.show()

print("\n--- Continuous Variable Statistics ---")
print(df[stat_cols].describe().round(2).loc[["mean", "std", "min", "max"]])

```



--- Continuous Variable Statistics ---

	FTHG	FTAG	HS	AS	HST	AST	HC	AC	HF	AF
mean	1.62	1.38	14.31	11.83	4.92	4.19	5.64	4.73	10.72	11.21
std	1.28	1.19	5.68	5.20	2.58	2.35	3.06	2.90	3.45	3.53
min	0.00	0.00	1.00	1.00	0.00	0.00	0.00	0.00	2.00	1.00
max	7.00	8.00	36.00	37.00	16.00	15.00	17.00	18.00	21.00	22.00

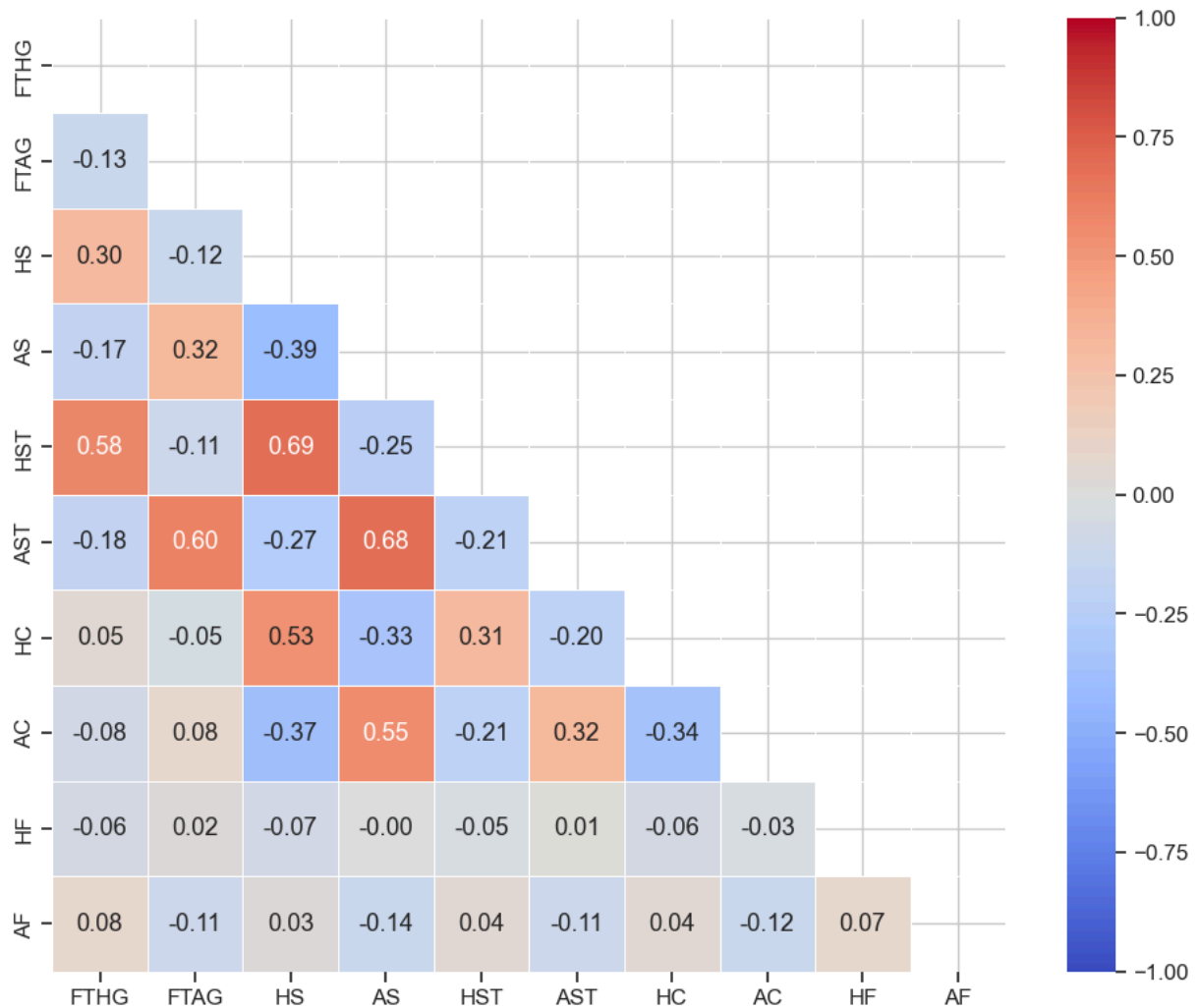
In [5]: fig, axes = plt.subplots(1, 1, figsize=(10, 8))

```

corr = df[stat_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True, linewidths=0.5)

plt.tight_layout()
plt.show()

```



```
In [6]: home_goals, away_goals = df["FTHG"], df["FTAG"]
lam_h, lam_a = home_goals.mean(), away_goals.mean()
x_vals = np.arange(0, 9)

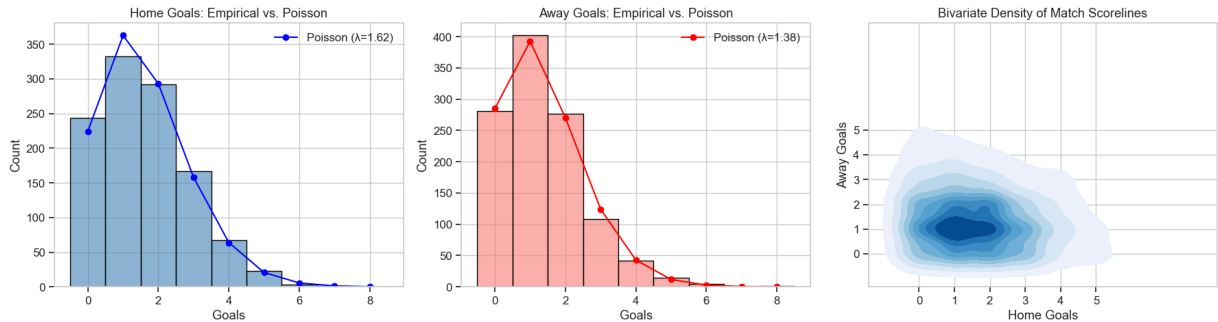
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Home Poisson fit
sns.histplot(home_goals, bins=np.arange(0, 10) - 0.5, stat="count",
              alpha=0.6, color="steelblue", ax=axes[0])
axes[0].plot(x_vals, poisson.pmf(x_vals, lam_h) * len(df),
              "bo-", label=f"Poisson (λ={lam_h:.2f})")
axes[0].set_title("Home Goals: Empirical vs. Poisson")
axes[0].set_xlabel("Goals")
axes[0].legend()

# Away Poisson fit
sns.histplot(away_goals, bins=np.arange(0, 10) - 0.5, stat="count",
              alpha=0.6, color="salmon", ax=axes[1])
axes[1].plot(x_vals, poisson.pmf(x_vals, lam_a) * len(df),
              "ro-", label=f"Poisson (λ={lam_a:.2f})")
axes[1].set_title("Away Goals: Empirical vs. Poisson")
axes[1].set_xlabel("Goals")
axes[1].legend()
```

```
# Bivariate scoreline density
sns.kdeplot(x=df["FTHG"], y=df["FTAG"], cmap="Blues", fill=True, bw_adjust=1)
axes[2].set_title("Bivariate Density of Match Scorelines")
axes[2].set_xlabel("Home Goals")
axes[2].set_ylabel("Away Goals")
axes[2].set_xticks(range(6))
axes[2].set_yticks(range(6))

plt.tight_layout()
plt.show()
```



```
In [7]: df["Shot_Diff"] = df["HS"] - df["AS"]
df["Home_Win"] = (df["FTR"] == "H").astype(int)

bins = pd.qcut(df["Shot_Diff"], q=10, duplicates="drop")
prob_df = df.groupby(bins, observed=True)["Home_Win"].mean().reset_index()
prob_df["Bin_Center"] = prob_df["Shot_Diff"].apply(lambda x: x.mid).astype(float)

# Compute normalised bookmaker probabilities
df["Overround"] = 1/df["B365H"] + 1/df["B365D"] + 1/df["B365A"]
df["Prob_H"] = (1/df["B365H"]) / df["Overround"]
df["Prob_D"] = (1/df["B365D"]) / df["Overround"]
df["Prob_A"] = (1/df["B365A"]) / df["Overround"]
df["Actual_H"] = (df["FTR"] == "H").astype(int)
df["Actual_D"] = (df["FTR"] == "D").astype(int)
df["Actual_A"] = (df["FTR"] == "A").astype(int)

y = df["FTR"].map({"A": 0, "D": 1, "H": 2}).astype(int)
market_loss = log_loss(
    y, np.column_stack([df["Prob_A"], df["Prob_D"], df["Prob_H"]]), labels=L
)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

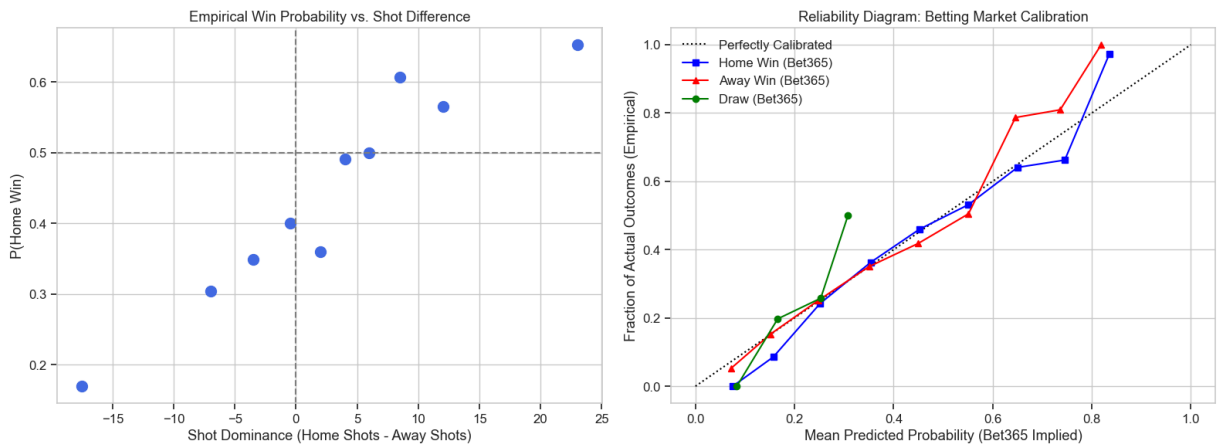
axes[0].scatter(prob_df["Bin_Center"], prob_df["Home_Win"], s=100, color="red")
axes[0].axhline(0.5, color="gray", linestyle="--")
axes[0].axvline(0, color="gray", linestyle="--")
axes[0].set_title("Empirical Win Probability vs. Shot Difference")
axes[0].set_xlabel("Shot Dominance (Home Shots - Away Shots)")
axes[0].set_ylabel("P(Home Win)")

axes[1].plot([0, 1], [0, 1], "k:", label="Perfectly Calibrated")
for probs_col, actuals_col, label, color, marker in [
    ("Prob_H", "Actual_H", "Home Win (Bet365)", "blue", "s"),
    ("Prob_A", "Actual_A", "Away Win (Bet365)", "red", "^"),
    ("Prob_D", "Actual_D", "Draw (Bet365)", "green", "o"),
]
```

```
]:
fp, mp = calibration_curve(df[actuals_col], df[probs_col], n_bins=10)
axes[1].plot(mp, fp, f"{marker}-", color=color, label=label)

axes[1].set_title("Reliability Diagram: Betting Market Calibration")
axes[1].set_xlabel("Mean Predicted Probability (Bet365 Implied)")
axes[1].set_ylabel("Fraction of Actual Outcomes (Empirical)")
axes[1].legend(loc="upper left")
axes[1].set_xlim(-0.05, 1.05)
axes[1].set_ylim(-0.05, 1.05)

plt.tight_layout()
plt.show()
print(f"Bet365 market log-loss: {market_loss:.4f}")
```



Bet365 market log-loss: 0.9639

```
In [8]: TARGET_TEAM = "Man City" # change to any team in the dataset

tm = df[(df["HomeTeam"] == TARGET_TEAM) | (df["AwayTeam"] == TARGET_TEAM)].c
tm["Team_Scored"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["FTHG"], tm
tm["Team_Conceded"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["FTAG"], tm
tm["Team_Shots"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["HS"], tm
tm["Opp_Shots"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["AS"], tm
tm = tm.reset_index(drop=True)
tm["Match_No"] = tm.index + 1
tm["Rolling_Shots"] = tm["Team_Shots"].rolling(WINDOW, min_periods=1).m
tm["Rolling_Opp_Shots"] = tm["Opp_Shots"].rolling(WINDOW, min_periods=1).me

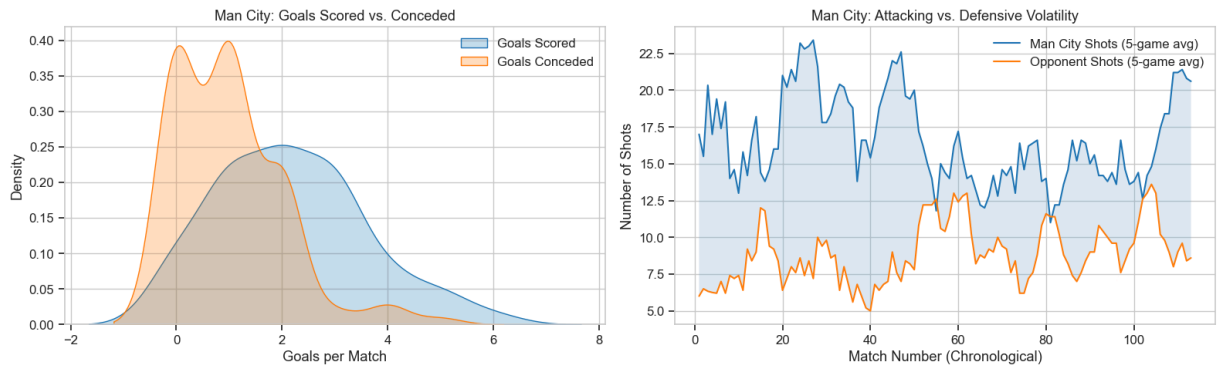
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

sns.kdeplot(data=tm["Team_Scored"], fill=True, label="Goals Scored", ax=
sns.kdeplot(data=tm["Team_Conceded"], fill=True, label="Goals Conceded", ax=
axes[0].set_title(f"{TARGET_TEAM}: Goals Scored vs. Conceded")
axes[0].set_xlabel("Goals per Match")
axes[0].legend()

axes[1].plot(tm["Match_No"], tm["Rolling_Shots"], label=f"{TARGET_TEAM} S
axes[1].plot(tm["Match_No"], tm["Rolling_Opp_Shots"], label=f"Opponent Shots
axes[1].fill_between(tm["Match_No"],
                    tm["Rolling_Shots"], tm["Rolling_Opp_Shots"], alpha=0.1
axes[1].set_title(f"{TARGET_TEAM}: Attacking vs. Defensive Volatility")
axes[1].set_xlabel("Match Number (Chronological)")
```

```
axes[1].set_ylabel("Number of Shots")
axes[1].legend()

plt.tight_layout()
plt.show()
```



```
In [9]: # — Stadium GPS coordinates —————
STADIUM_COORDS = {
    "Arsenal": (51.5549, -0.1084),
    "Aston Villa": (52.5092, -1.8847),
    "Bournemouth": (50.7352, -1.8380),
    "Brentford": (51.4907, -0.2884),
    "Brighton": (50.8618, -0.0861),
    "Burnley": (53.7889, -2.2298),
    "Chelsea": (51.4817, -0.1910),
    "Crystal Palace": (51.3983, -0.0855),
    "Everton": (53.4388, -2.9664),
    "Fulham": (51.4749, -0.2217),
    "Ipswich": (52.0536, 1.1444),
    "Leeds": (53.7778, -1.5737),
    "Leicester": (52.6204, -1.1422),
    "Liverpool": (53.4308, -2.9608),
    "Luton": (51.8836, -0.4318),
    "Man City": (53.4831, -2.2004),
    "Man United": (53.4631, -2.2913),
    "Newcastle": (54.9756, -1.6218),
    "Nott'm Forest": (52.9399, -1.1327),
    "Sheffield United": (53.3703, -1.4705),
    "Southampton": (50.9058, -1.3911),
    "Sunderland": (54.9146, -1.3883),
    "Tottenham": (51.6043, -0.0665),
    "West Ham": (51.5386, -0.0163),
    "Wolves": (52.5904, -2.1302),
}

# — Helper functions —————
def haversine_miles(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2])
    a = sin((lat2 - lat1) / 2)**2 + cos(lat1) * cos(lat2) * sin((lon2 - lon1) / 2)**2
    return 3958.8 * 2 * atan2(sqrt(a), sqrt(1 - a))

def build_team_log(df):
    """Reshape wide match data to one row per team per match."""
```

```

home = df[["Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR", "HST",
home.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
home["Venue"] = "H"
home["Points"] = home["FTR"].map({"H": 3, "D": 1, "A": 0})

away = df[["Date", "AwayTeam", "HomeTeam", "FTAG", "FTHG", "FTR", "AST",
away.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
away["Venue"] = "A"
away["Points"] = away["FTR"].map({"A": 3, "D": 1, "H": 0})

return (
    pd.concat([home, away], ignore_index=True)
    .sort_values(["Team", "Date"])
    .reset_index(drop=True)
)

def venue_expanding_mean(group_df, venue_flag, col):
    """
    Leakage-free expanding average restricted to one venue type.
    Rows at the wrong venue become NaN; expanding().mean() skips them.
    Shift(1) ensures the current match is never included.
    """
    mask = group_df["Venue"] == venue_flag
    return (
        group_df[col]
        .where(mask)
        .expanding()
        .mean()
        .shift(1)
        .fillna(0)
    )

def compute_travel_fatigue(group_df, team_name):
    """Cumulative travel miles over the last WINDOW matches (leakage-free)."""
    if team_name not in STADIUM_COORDS:
        return pd.Series(0.0, index=group_df.index)

    hlat, hlon = STADIUM_COORDS[team_name]
    prev_lat, prev_lon = hlat, hlon
    dists = []

    for _, row in group_df.iterrows():
        clat, clon = (
            (hlat, hlon)
            if row["Venue"] == "H"
            else STADIUM_COORDS.get(row["Opponent"], (hlat, hlon))
        )
        dists.append(haversine_miles(prev_lat, prev_lon, clat, clon))
        prev_lat, prev_lon = clat, clon

    return (
        pd.Series(dists, index=group_df.index)
        .shift(1)
        .rolling(WINDOW, min_periods=1)
    )

```

```

        .sum()
        .fillna(0)
    )

# — Build team log —
print("Building per-team match log...")
team_log = build_team_log(df)
grp = team_log.groupby("Team")

# — Overall rolling form (leakage-free via shift inside transform) —
print("Computing rolling form...")
team_log["Roll_GF_5"] = grp["GF"].transform(
    lambda s: s.shift(1).rolling(WINDOW, min_periods=1).mean().fillna(0)
)
team_log["Roll_SOT_5"] = grp["SOT_F"].transform(
    lambda s: s.shift(1).rolling(WINDOW, min_periods=1).mean().fillna(0)
)
team_log["Roll_Pts_5"] = grp["Points"].transform(
    lambda s: s.shift(1).rolling(WINDOW, min_periods=1).sum().fillna(0)
)

# — Venue-specific expanding form (loop avoids pandas apply index issues) —
print("Computing venue-specific form...")
for col_out, venue_flag, col_in in [
    ("Home_GF_Form", "H", "GF"),
    ("Home_SOT_Form", "H", "SOT_F"),
    ("Away_GF_Form", "A", "GF"),
    ("Away_SOT_Form", "A", "SOT_F"),
]:
    team_log[col_out] = 0.0
    for team, g in team_log.groupby("Team"):
        team_log.loc[g.index, col_out] = venue_expanding_mean(g, venue_flag,

# — Travel fatigue —
print("Computing travel fatigue...")
team_log["Travel_Fatigue_5"] = 0.0
for team, g in team_log.groupby("Team"):
    team_log.loc[g.index, "Travel_Fatigue_5"] = compute_travel_fatigue(g, te

# — Merge features back to match level —
print("Merging to match level...")
FEAT_COLS = [
    "Date", "Team",
    "Roll_GF_5", "Roll_SOT_5", "Roll_Pts_5",
    "Home_GF_Form", "Home_SOT_Form",
    "Away_GF_Form", "Away_SOT_Form",
    "Travel_Fatigue_5",
]

home_feats = (
    team_log[team_log["Venue"] == "H"][FEAT_COLS]
    .rename(columns={"Team": "HomeTeam",
                    **{c: f"H_{c}" for c in FEAT_COLS if c not in ("Date",
)
)
away_feats = (
    team_log[team_log["Venue"] == "A"][FEAT_COLS]
    .rename(columns={"Team": "AwayTeam",
                    **{c: f"A_{c}" for c in FEAT_COLS if c not in ("Date",

```

```

)

df = df.merge(home_feats, on=["Date", "HomeTeam"], how="left")
df = df.merge(away_feats, on=["Date", "AwayTeam"], how="left")

# — Final feature list & target —————
FEATURES = [
    "H_Roll_GF_5", "H_Roll_SOT_5", "H_Home_GF_Form", "H_Home_SOT_Form",
    "H_Roll_Pts_5", "H_Travel_Fatigue_5",
    "A_Roll_GF_5", "A_Roll_SOT_5", "A_Away_GF_Form", "A_Away_SOT_Form",
    "A_Roll_Pts_5", "A_Travel_Fatigue_5",
]

X = df[FEATURES].fillna(0)
y = df["FTR"].map({"A": 0, "D": 1, "H": 2}).astype(int)
N_FEAT = X.shape[1] # 12

print(f"\nFeature matrix : {X.shape}")
print(f"Target counts : \n{y.value_counts().sort_index().rename({0: 'Away', 1: 'Draw', 2: 'Home'})}")
print("\n--- Feature Summary ---")
print(X.describe().round(3).loc[["mean", "std", "min", "max"]])

```

Building per-team match log...
 Computing rolling form...
 Computing venue-specific form...
 Computing travel fatigue...
 Merging to match level...

Feature matrix : (1130, 12)

Target counts :

FTR

Away 366

Draw 276

Home 488

Name: count, dtype: int64

--- Feature Summary ---

	H_Roll_GF_5	H_Roll_SOT_5	H_Home_GF_Form	H_Home_SOT_Form	\
mean	1.463	4.469	1.663	5.051	
std	0.673	1.455	0.644	1.556	
min	0.000	0.000	0.000	0.000	
max	4.800	10.000	5.000	13.000	

	H_Roll_Pts_5	H_Travel_Fatigue_5	A_Roll_GF_5	A_Roll_SOT_5	\
mean	6.589	501.010	1.500	4.582	
std	3.594	223.386	0.678	1.489	
min	0.000	0.000	0.000	0.000	
max	15.000	1251.966	5.000	13.000	

	A_Away_GF_Form	A_Away_SOT_Form	A_Roll_Pts_5	A_Travel_Fatigue_5
mean	1.39	4.234	6.810	483.086
std	0.50	1.265	3.583	226.751
min	0.00	0.000	0.000	0.000
max	4.00	10.667	15.000	1247.894

```

In [10]: # Note: solver='saga' replaces 'lbfgs' for LogisticRegression – 'lbfgs' was
# producing hundreds of divide-by-zero/overflow warnings from sklearn's matn
# on this dataset. 'saga' is numerically more stable for multiclass problems
#
# Note on LSTM architecture: input_shape=(1, N_FEAT) treats each match as a
# single timestep. This replicates the original design. True sequential
# modelling would pass a sliding window of prior matches as the time dimensi

BASE_MODELS = {
    "1. Logistic Regression": LogisticRegression(
        solver="saga", max_iter=2000, random_state=SEED),
    "2. Random Forest": RandomForestClassifier(random_state=SEED),
    "3. Gradient Boosting": GradientBoostingClassifier(random_state=SEED),
    "4. XGBoost": XGBClassifier(
        objective="multi:softprob", eval_metric="mlogloss",
        random_state=SEED, verbosity=0),
    "5. LightGBM": LGBMClassifier(random_state=SEED, verbose=-1),
    "6. Neural Network (MLP)": MLPClassifier(
        solver="adam", max_iter=500, early_stopping=True, random_state=SEED)
}

PARAM_GRIDS = {
    "1. Logistic Regression": {
        "model__C": [0.01, 0.1, 1.0],
        "model__penalty": ["l2"],
    },
    "2. Random Forest": {
        "model__n_estimators": [100],
        "model__max_depth": [5, 10, None],
    },
    "3. Gradient Boosting": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__max_depth": [3, 5],
    },
    "4. XGBoost": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__max_depth": [3, 5],
    },
    "5. LightGBM": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__num_leaves": [31, 63],
    },
    "6. Neural Network (MLP)": {
        "model__hidden_layer_sizes": [(32,), (64, 32)],
        "model__alpha": [0.001, 0.01],
    },
}

def build_lstm(n_features):
    """Build and compile the LSTM used throughout this notebook."""
    model = Sequential([

```

```

        LSTM(32, input_shape=(1, n_features), activation="relu"),
        Dropout(0.2),
        Dense(16, activation="relu"),
        Dense(3, activation="softmax"),
    ])
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")
    return model

```

```
print("Model definitions ready.")
```

Model definitions ready.

```

In [ ]: # --- Hyperparameter tuning ---
print("---- Hyperparameter Tuning ----\n")
tuned_models = {}
for name, base_model in BASE_MODELS.items():
    pipe = Pipeline([("scaler", StandardScaler()), ("model", base_model)])
    grid = GridSearchCV(
        estimator=pipe, param_grid=PARAM_GRIDS[name],
        cv=TSCV, scoring="neg_log_loss", n_jobs=-1, refit=True,
    )
    grid.fit(X, y)
    tuned_models[name] = grid.best_estimator_
    print(f"[{name}] best log-loss: {-grid.best_score_:.4f}")

# --- Walk-forward evaluation ---
print("\n---- Walk-Forward Evaluation ----\n")
results = {}
poisson_y_true, poisson_y_prob = [], [] # Saved for Poisson confusion matrix

# 1. Standard sklearn models
for name, pipe in tuned_models.items():
    f_logloss, f_acc, f_f1 = [], [], []
    for train_idx, test_idx in TSCV.split(X):
        pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
        probs = pipe.predict_proba(X.iloc[test_idx])
        preds = np.argmax(probs, axis=1)
        y_test = y.iloc[test_idx]

        f_logloss.append(log_loss(y_test, probs, labels=LABELS))
        f_acc.append(accuracy_score(y_test, preds))
        f_f1.append(f1_score(y_test, preds, average='macro'))

    results[name] = {'Log-Loss': np.mean(f_logloss), 'Accuracy': np.mean(f_acc), 'F1': np.mean(f_f1)}

# 2. Poisson regression
f_logloss, f_acc, f_f1 = [], [], []
for train_idx, test_idx in TSCV.split(X):
    sc = StandardScaler()
    X_tr, X_te = sc.fit_transform(X.iloc[train_idx]), sc.transform(X.iloc[test_idx])

    pr_h = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_tr, df["FTHG"].iloc[train_idx])
    pr_a = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_tr, df["FTAG"].iloc[train_idx])

    probs = np.zeros((len(test_idx), 3))

```

```

for i, (lh, la) in enumerate(zip(pr_h.predict(X_te), pr_a.predict(X_te))):
    lh, la = max(float(lh), 1e-6), max(float(la), 1e-6)
    ph, pd_tie, pa = 0.0, 0.0, 0.0
    for h in range(10):
        for a in range(10):
            ps = poisson.pmf(h, lh) * poisson.pmf(a, la)
            if h > a: ph += ps
            elif h == a: pd_tie += ps
            else: pa += ps
    tot = ph + pd_tie + pa
    probs[i] = [pa / tot, pd_tie / tot, ph / tot]

preds = np.argmax(probs, axis=1)
y_test = y.iloc[test_idx]

f_logloss.append(log_loss(y_test, probs, labels=LABELS))
f_acc.append(accuracy_score(y_test, preds))
f_f1.append(f1_score(y_test, preds, average='macro'))

poisson_y_true.extend(y_test.tolist())
poisson_y_prob.extend(probs.tolist())

results["7. Poisson Regression"] = {'Log-Loss': np.mean(f_logloss), 'Accuracy': np.mean(f_acc), 'F1-Macro': np.mean(f_f1)}

# 3. RNN (LSTM)
f_logloss, f_acc, f_f1 = [], [], []
for train_idx, test_idx in TSCV.split(X):
    sc = StandardScaler()
    X_tr = sc.fit_transform(X.iloc[train_idx]).reshape(-1, 1, N_FEAT)
    X_te = sc.transform(X.iloc[test_idx]).reshape(-1, 1, N_FEAT)

    lstm = build_lstm(N_FEAT)
    lstm.fit(X_tr, y.iloc[train_idx], epochs=40, batch_size=16, verbose=0)

    probs = lstm.predict(X_te, verbose=0)
    preds = np.argmax(probs, axis=1)
    y_test = y.iloc[test_idx]

    f_logloss.append(log_loss(y_test, probs, labels=LABELS))
    f_acc.append(accuracy_score(y_test, preds))
    f_f1.append(f1_score(y_test, preds, average='macro'))

results["8. RNN (LSTM)"] = {'Log-Loss': np.mean(f_logloss), 'Accuracy': np.mean(f_acc), 'F1-Macro': np.mean(f_f1)}

# 4. Market Baseline
m_preds = np.argmax(np.column_stack([df["Prob_A"], df["Prob_D"], df["Prob_H"]]), axis=1)
results["Bet365 (Baseline)"] = {
    'Log-Loss': market_loss,
    'Accuracy': accuracy_score(y, m_preds),
    'F1-Macro': f1_score(y, m_preds, average='macro')
}

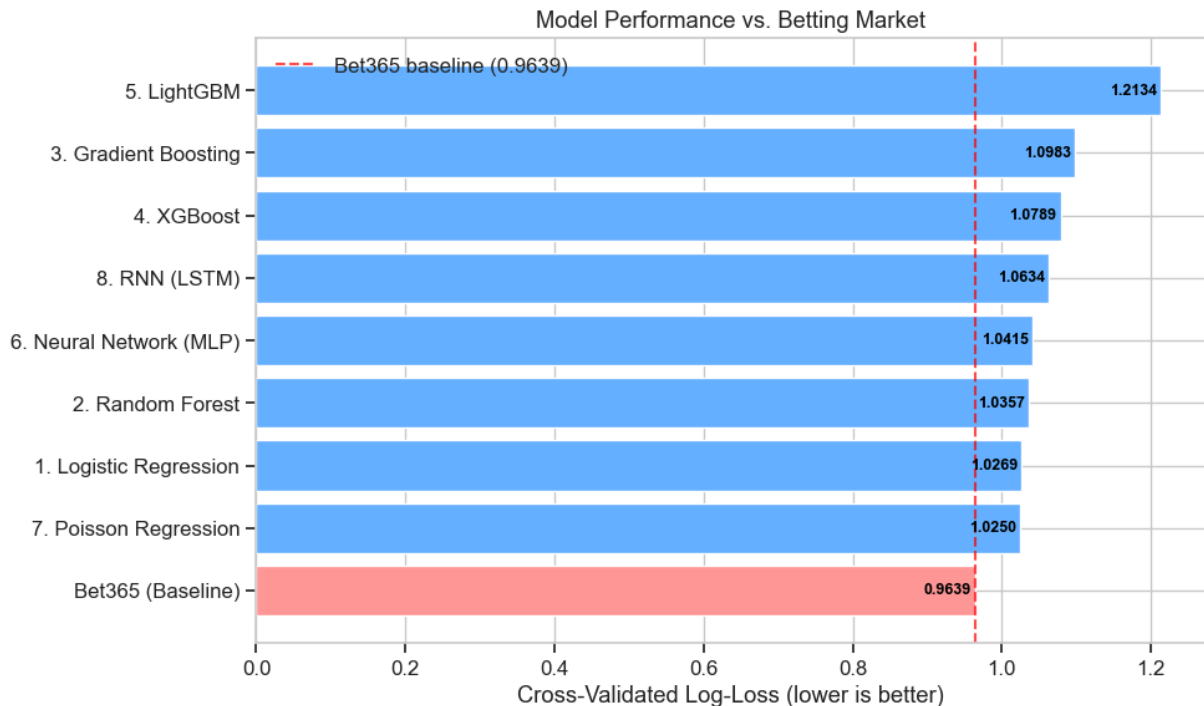
# — Print ranking —
print(f"\n{'Model':<30} | {'Log-Loss ↓':<10} | {'Accuracy ↑':<10} | {'F1-Macro ↑':<10}")
print("-" * 68)
# Sort models by Log-Loss (lowest is best)

```

```
for name, metrics in sorted(results.items(), key=lambda x: x[1]['Log-Loss'])
    marker = " <--" if "Bet365" in name else ""
    print(f"{name:<30} | {metrics['Log-Loss']:.4f}      | {metrics['Accuracy']
```

```
In [17]: sorted_res = dict(sorted(results.items(), key=lambda x: x[1]['Log-Loss']))
colors = ["#ff9999" if "Bet365" in n else "#66b3ff" for n in sorted_res]

plt.figure(figsize=(10, 6))
log_losses = [metrics["Log-Loss"] for metrics in sorted_res.values()]
bars = plt.barh(list(sorted_res.keys()), log_losses,
                color=colors, edgecolor="white")
plt.axvline(market_loss, color="red", linestyle="--", alpha=0.7,
            label=f"Bet365 baseline ({market_loss:.4f})")
for bar in bars:
    plt.text(bar.get_width() - 0.005,
             bar.get_y() + bar.get_height() / 2,
             f"{bar.get_width():.4f}",
             va="center", ha="right", color="black", fontweight="bold", font
plt.xlabel("Cross-Validated Log-Loss (lower is better)")
plt.title("Model Performance vs. Betting Market")
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [13]: # — Poisson Regression Coefficients —————
print("==== 7. Poisson Regression — Feature Coefficients =====")
sc_full = StandardScaler()
X_sc     = sc_full.fit_transform(X)

pr_h_full = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_sc, df["FTHG"])
pr_a_full = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_sc, df["FTAG"])

coef_df = pd.DataFrame({
    "Feature": FEATURES,
```

```

    "Home Goals ( $\lambda$ ) Coef": pr_h_full.coef_,
    "Away Goals ( $\lambda$ ) Coef": pr_a_full.coef_
})

# We'll sort by the absolute impact on Home Goals to see what drives the mat
coef_df["Abs_Home_Impact"] = coef_df["Home Goals ( $\lambda$ ) Coef"].abs()
coef_df = coef_df.sort_values("Abs_Home_Impact", ascending=False).drop(column

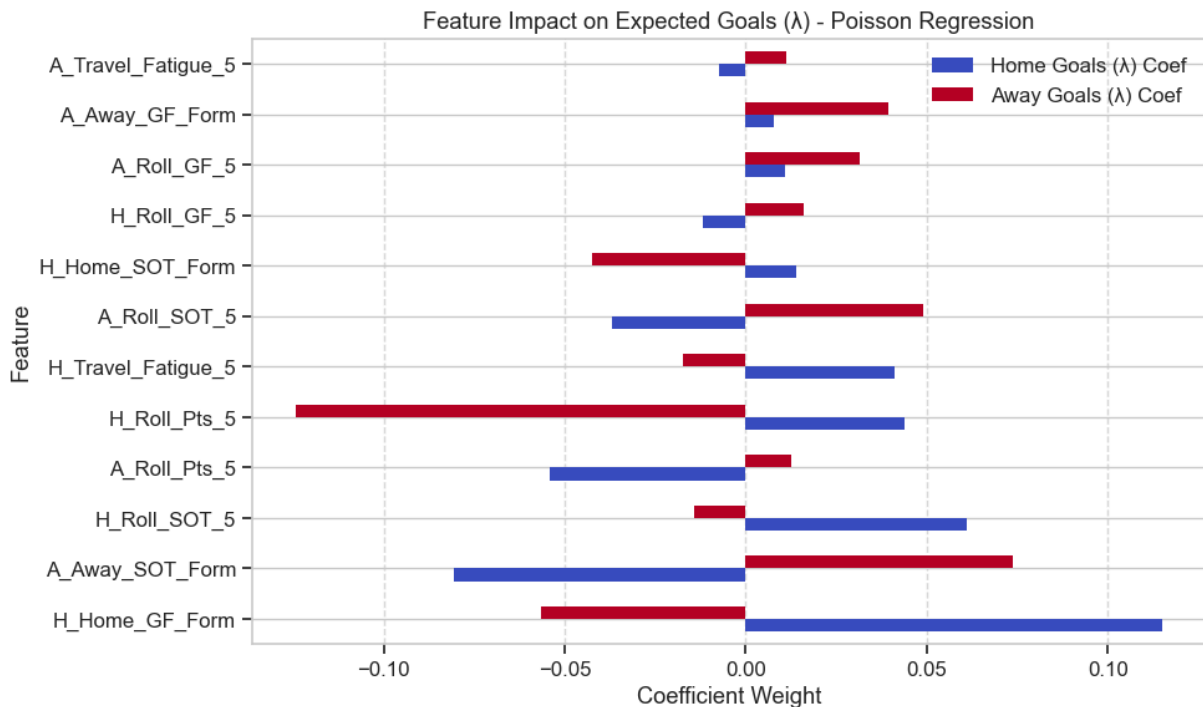
print(coef_df.to_string(index=False))

# Plot the coefficients for visual comparison
coef_df.set_index("Feature").plot(kind="barh", figsize=(10, 6), colormap="cc
plt.title("Feature Impact on Expected Goals ( $\lambda$ ) - Poisson Regression")
plt.xlabel("Coefficient Weight")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```

==== 7. Poisson Regression – Feature Coefficients ====

Feature	Home Goals (λ) Coef	Away Goals (λ) Coef
H_Home_GF_Form	0.114838	-0.056506
A_Away_SOT_Form	-0.080623	0.073888
H_Roll_SOT_5	0.060914	-0.014280
A_Roll_Pts_5	-0.054022	0.012668
H_Roll_Pts_5	0.043873	-0.124223
H_Travel_Fatigue_5	0.041199	-0.017433
A_Roll_SOT_5	-0.036991	0.048991
H_Home_SOT_Form	0.013984	-0.042373
H_Roll_GF_5	-0.011924	0.015930
A_Roll_GF_5	0.010809	0.031424
A_Away_GF_Form	0.007851	0.039434
A_Travel_Fatigue_5	-0.007261	0.011224



```

In [14]: poisson_y_true_arr = np.array(poisson_y_true)
         poisson_y_prob_arr = np.array(poisson_y_prob)

```

```

poisson_preds      = np.argmax(poisson_y_prob_arr, axis=1)
class_labels       = ["Away Win", "Draw", "Home Win"]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

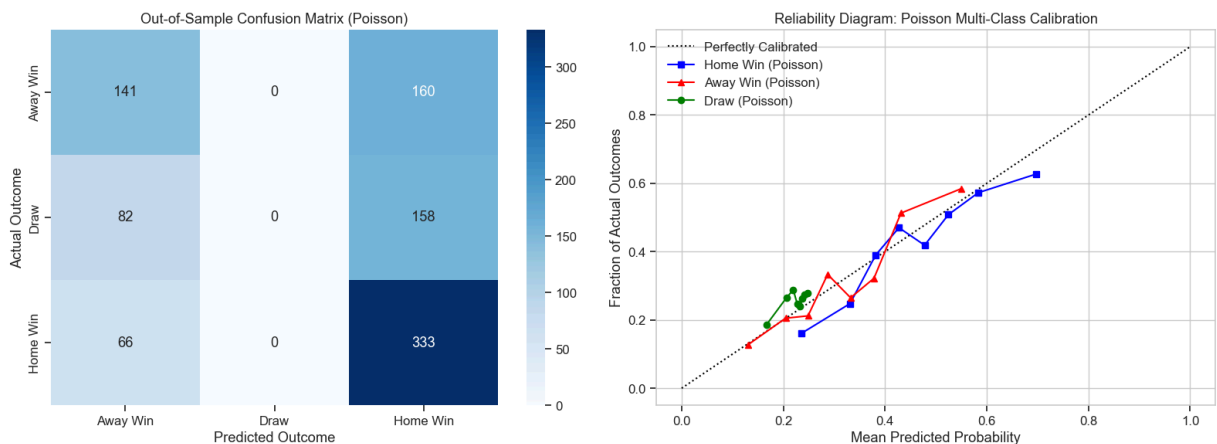
# Confusion matrix
cm = confusion_matrix(poisson_y_true_arr, poisson_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_labels, yticklabels=class_labels, ax=axes[0])
axes[0].set_title("Out-of-Sample Confusion Matrix (Poisson)")
axes[0].set_xlabel("Predicted Outcome")
axes[0].set_ylabel("Actual Outcome")

# Calibration curve
axes[1].plot([0, 1], [0, 1], "k:", label="Perfectly Calibrated")
for cls_idx, label, color, marker in [
    (2, "Home Win (Poisson)", "blue", "s"),
    (0, "Away Win (Poisson)", "red", "^"),
    (1, "Draw (Poisson)", "green", "o"),
]:
    actual = (poisson_y_true_arr == cls_idx).astype(int)
    fp, mp = calibration_curve(actual, poisson_y_prob_arr[:, cls_idx],
                              n_bins=8, strategy="quantile")
    axes[1].plot(mp, fp, f"{marker}-", color=color, label=label)

axes[1].set_title("Reliability Diagram: Poisson Multi-Class Calibration")
axes[1].set_xlabel("Mean Predicted Probability")
axes[1].set_ylabel("Fraction of Actual Outcomes")
axes[1].legend(loc="upper left")
axes[1].set_xlim(-0.05, 1.05)
axes[1].set_ylim(-0.05, 1.05)

plt.tight_layout()
plt.show()

```



```

In [15]: FINAL_WEEK = [
    {"HomeTeam": "Man City", "AwayTeam": "Aston Villa"},
    {"HomeTeam": "Brighton", "AwayTeam": "Man United"},
    {"HomeTeam": "Fulham", "AwayTeam": "Newcastle"},
    {"HomeTeam": "Tottenham", "AwayTeam": "Everton"},
    {"HomeTeam": "Liverpool", "AwayTeam": "Brentford"},
    {"HomeTeam": "Burnley", "AwayTeam": "Wolves"},

```

```

{"HomeTeam": "Nott'm Forest", "AwayTeam": "Bournemouth"},
{"HomeTeam": "West Ham", "AwayTeam": "Leeds"},
{"HomeTeam": "Sunderland", "AwayTeam": "Chelsea"},
{"HomeTeam": "Crystal Palace", "AwayTeam": "Arsenal"},
]

def get_latest_features(team: str, is_home: bool) -> dict:
    """Return the most recent rolling features for a team from team_log."""
    hist = team_log[team_log["Team"] == team].tail(1)
    p = "H_" if is_home else "A_"
    vp = "Home_" if is_home else "Away_"
    zeros = {
        f"{p}Roll_GF_5": 0.0, f"{p}Roll_SOT_5": 0.0, f"{p}Roll_Pts_5": 0.0,
        f"{p}{vp}GF_Form": 0.0, f"{p}{vp}SOT_Form": 0.0,
        f"{p}Travel_Fatigue_5": 0.0,
    }
    if hist.empty:
        return zeros
    r = hist.iloc[0]
    return {
        f"{p}Roll_GF_5": r["Roll_GF_5"],
        f"{p}Roll_SOT_5": r["Roll_SOT_5"],
        f"{p}Roll_Pts_5": r["Roll_Pts_5"],
        f"{p}{vp}GF_Form": r[f"{vp}GF_Form"],
        f"{p}{vp}SOT_Form": r[f"{vp}SOT_Form"],
        f"{p}Travel_Fatigue_5": r["Travel_Fatigue_5"],
    }

# Train final Poisson models on all available data
print("Training deployment Poisson Regression on full dataset...")
sc_deploy = StandardScaler()
X_all_sc = sc_deploy.fit_transform(X)

pr_h_deploy = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_all_sc, df["F
pr_a_deploy = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_all_sc, df["F

# Build live feature matrix (column order must match FEATURES)
X_live = pd.DataFrame([
    {**get_latest_features(m["HomeTeam"], True),
     **get_latest_features(m["AwayTeam"], False)}
    for m in FINAL_WEEK
])[FEATURES].fillna(0)

X_live_sc = sc_deploy.transform(X_live)

# Calculate bivariate Poisson probabilities for live matches
live_preds = []
for lh, la in zip(pr_h_deploy.predict(X_live_sc), pr_a_deploy.predict(X_live
    lh, la = max(float(lh), 1e-6), max(float(la), 1e-6)
    ph, pd_tie, pa = 0.0, 0.0, 0.0
    for h in range(10):
        for a in range(10):
            ps = poisson.pmf(h, lh) * poisson.pmf(a, la)
            if h > a: ph += ps
            elif h == a: pd_tie += ps

```

```

        else:          pa += ps
        tot = ph + pd_tie + pa
        live_preds.append((pa / tot, pd_tie / tot, ph / tot))

# Print predictions
print(f"\n--- Matchday 38 (2025/26) Predictions – Poisson Regression ---\n")
header = f"{'Fixture':<35} | {'Home %':>7} | {'Draw %':>7} | {'Away %':>7} |"
print(header)
print("-" * len(header))
for m, (pa, pd_tie, ph) in zip(FINAL_WEEK, live_preds):
    # Clamp to avoid division by zero in implied odds
    ph, pd_tie, pa = max(float(ph), 1e-4), max(float(pd_tie), 1e-4), max(float(pa), 1e-4)
    fixture = f"{m['HomeTeam']} vs {m['AwayTeam']}"
    print(
        f"{'fixture':<35} | {ph*100:>6.1f}% | {pd_tie*100:>6.1f}% | {pa*100:>6.1f}% |"
        f" | {1/ph:>7.2f} | {1/pd_tie:>7.2f} | {1/pa:>7.2f}"
    )

```

Training deployment Poisson Regression on full dataset...

--- Matchday 38 (2025/26) Predictions – Poisson Regression ---

Fixture	Home %	Draw %	Away %	H Odds
D Odds A Odds				

Man City vs Aston Villa	64.8%	19.6%	15.5%	1.54
5.09 6.44				
Brighton vs Man United	51.4%	23.5%	25.1%	1.94
4.25 3.99				
Fulham vs Newcastle	35.7%	24.7%	39.6%	2.80
4.05 2.52				
Tottenham vs Everton	52.1%	23.0%	24.9%	1.92
4.36 4.02				
Liverpool vs Brentford	54.6%	23.1%	22.4%	1.83
4.33 4.47				
Burnley vs Wolves	36.0%	24.8%	39.1%	2.77
4.03 2.56				
Nott'm Forest vs Bournemouth	36.6%	25.5%	37.9%	2.74
3.92 2.64				
West Ham vs Leeds	34.5%	25.3%	40.3%	2.90
3.96 2.48				
Sunderland vs Chelsea	42.1%	24.4%	33.5%	2.38
4.10 2.98				
Crystal Palace vs Arsenal	30.9%	23.7%	45.3%	3.23
4.21 2.21				

Fixture (Home vs Away) Home Win % Draw % Away Win %

Man City vs Aston Villa 64.5% 19.3% 16.2%

Brighton vs Man United 50.5% 23.5% 26.0%

Fulham vs Newcastle 33.0% 25.6% 41.4%

Tottenham vs Everton 50.8% 24.7% 24.5%

Liverpool vs Brentford 54.0% 23.0% 23.0%

Burnley vs Wolves 37.9% 26.9% 35.2%

Nott'm Forest vs Bournemouth 29.2% 25.3% 45.5%

West Ham vs Leeds 50.7% 25.6% 23.7%

Sunderland vs Chelsea 26.4% 25.4% 48.2%

Crystal Palace vs Arsenal 24.6% 24.0% 51.4%